

Resumen

La eliminación de datos es una operación fundamental en cualquier aplicación que maneja información. En el contexto de desarrollo web, implementar correctamente la funcionalidad de borrado es esencial para mantener la integridad de los datos y proporcionar una experiencia de usuario completa. En esta ocasión, exploraremos cómo implementar la operación DELETE en nuestro CRUD (Create, Read, Update, Delete) utilizando Node.js y el manejo de archivos para almacenamiento local.

¿Cómo implementar la operación DELETE en nuestro CRUD?

Para completar nuestro CRUD, necesitamos implementar la funcionalidad de eliminación de usuarios. Esta operación nos permitirá eliminar registros específicos de nuestra base de datos local basada en archivos. Vamos a crear un nuevo punto de entrada en nuestra aplicación que manejará las solicitudes DELETE.

Primero, creamos nuestro archivo para manejar la eliminación:

```
// app.delete.js
app.delete('/users/:id', (req, res) => {
  const userId = parseInt(req.params.id, 10);

  fs.readFile(usersFilePath, 'utf8', (err, data) => {
    if (err) {
      return res.status(500).json({ error: "Error con conexión de datos" });
    }

    let users = JSON.parse(data);
    users = users.filter(user => user.id !== userId);

    fs.writeFile(usersFilePath, JSON.stringify(users, null, 2), (err) => {
      if (err) {
        return res.status(500).json({ error: "Error al eliminar usuario" });
      }

      res.status(204).send();
    });
  });
});
```

En este código:

1. Creamos una ruta dinámica `/users/:id` que acepta el ID del usuario a eliminar
2. Extraemos el ID de los parámetros de la solicitud y lo convertimos a número
3. Leemos el archivo de usuarios existente
4. Filtramos la lista para excluir el usuario con el ID especificado
5. Escribimos la nueva lista de usuarios al archivo
6. Respondemos con un código 204 (No Content) para indicar éxito sin contenido de retorno

¿Cómo funciona el filtrado de usuarios?

La parte clave de nuestra implementación es el método `filter()` que utilizamos para eliminar el usuario específico:

```
users = users.filter(user => user.id !== userId);
```

Este método crea un nuevo array que incluye todos los elementos que pasan la condición especificada. En nuestro caso, estamos manteniendo todos los usuarios cuyo ID **no coincide** con el ID que queremos eliminar. De esta manera, efectivamente eliminamos el usuario deseado del array.

Es importante entender que este enfoque no modifica el array original, sino que crea uno nuevo sin el elemento que queremos eliminar. Esto sigue el principio de inmutabilidad que es una buena práctica en programación.

¿Cómo probamos nuestra funcionalidad de eliminación?

Para probar nuestra implementación, utilizamos Postman:

1. Creamos una nueva solicitud
2. Seleccionamos el método DELETE
3. Configuramos la URL con el ID del usuario que queremos eliminar (por ejemplo: `/users/2`)
4. No necesitamos enviar ningún cuerpo en la solicitud
5. Enviamos la solicitud y verificamos que recibimos un código de estado 204

Si todo funciona correctamente, el usuario con el ID especificado será eliminado de nuestro archivo de usuarios.

¿Cómo se integra DELETE con el resto de operaciones CRUD?

Con la implementación de DELETE, ahora tenemos un CRUD completo para nuestra aplicación:

- **Create:** Implementado con POST para agregar nuevos usuarios
- **Read:** Implementado con GET para obtener usuarios
- **Update:** Implementado con PUT para actualizar usuarios (con un reto pendiente)
- **Delete:** Implementado con DELETE para eliminar usuarios

Cada una de estas operaciones sigue un patrón similar:

1. Recibir una solicitud HTTP
2. Leer el archivo de datos existente
3. Realizar la operación correspondiente (crear, leer, actualizar o eliminar)
4. Escribir los cambios de vuelta al archivo (excepto en operaciones de lectura)
5. Responder con el código de estado y datos apropiados

¿Qué desafíos quedan pendientes?

Como se mencionó en la clase, existe un reto pendiente relacionado con la funcionalidad de actualización (PUT). El problema está relacionado con el manejo de IDs duplicados. La solución implica modificar la lógica de actualización para asegurarse de que:

1. Solo se actualice el usuario correcto cuando hay múltiples usuarios con el mismo ID
2. Se mantenga la integridad de los datos durante la actualización

Este es un excelente ejercicio para practicar la manipulación de datos y la lógica de programación.

¿Qué sigue después de implementar el CRUD completo?

Una vez que tenemos nuestro CRUD básico funcionando, podemos avanzar hacia conceptos más avanzados como:

1. Implementación de middlewares para manejar funcionalidades comunes
2. Configuración de sistemas de logging para rastrear actividades y errores
3. Mejora de la validación de datos
4. Implementación de autenticación y autorización
5. Optimización del rendimiento

Estos temas nos permitirán construir aplicaciones más robustas, seguras y mantenibles.

La implementación de un CRUD completo es un hito importante en el desarrollo de aplicaciones web. Con estas operaciones básicas, podemos construir sistemas complejos que permiten a los usuarios interactuar con datos de manera efectiva. ¿Has implementado alguna solución interesante para el reto de actualización? Comparte tus ideas y soluciones en los comentarios.

El problema de id lo resuelven los motores de bases de datos, pienso que una forma fácil de resolverlo es utilizar `some()`, pero esto lleva a leer el objeto completo, recorrerlo uno a uno hasta que la condición sea verdad, en definitiva, sería un desgaste y utilizar recursos a medida que los datos crecen...

La validación funciona para crear o eliminar.

```
async function deleteUser(req, res) {
  try {
    const userId = parseInt(req.params.id, 10);
    const data = await fs.readFile(usersFilePath, 'utf-8');

    let users = JSON.parse(data);

    const validation = users.some(user => user.id === userId)

    if(!validation){
      return res.status(500).json({
        status: "Id no existe, intenta eliminar otro id",
      })
    }

    users = users.filter(user => user.id !== userId);

    await fs.writeFile(usersFilePath, JSON.stringify(users, null, 2))

    res.status(204).end(); //204 no content, no regresa información
  } catch (error) {
    res.json({message: error.message})
  }
}
```